CARVE PARTY writeup

정명훈

1. 문제 정의

우선 이 문제는 화면에 나오는 호박을 클릭해서 잭오랜턴을 만드는 문제다. 화면에는 10,000번의 클릭이 남았다고 표시되고, 호박을 계속 클릭하면 눈, 코, 입이 만들어지면서 마지막에 flag를 확인할 수 있다.

따라서 가장 단순한 방법은 마우스로 10,000번 이상 클릭하는 것이지만, 나는 단순히 클릭만 해서 답을 얻기보다는 그 과정에서 flag가 어떻게 만들어지는지 알아보고 싶었다. 그래서 브라우저의 개발자 도구를 열고 JavaScript의 동작을 확인하기 시작했다.

2. 문제 탐색 및 JavaScript 코드 분석

먼저 코드를 대략 살펴보니 pumpkin이라는 배열과 클릭 횟수를 저장하는 counter, 연산에 사용되는 pie 변수가 있었다.

```
var pumpkin = [124, 112, 59, 73, 167, 100, 105, 75, 59, 23, 16,
               181, 165, 104, 43, 49, 118, 71, 112, 169, 43, 53];
var counter = 0;
var pie = 1;
```

여기서 예전에 JavaScript에서 배웠던 내용을 활용해 개발자 도구의 Console에서 counter 값을 직접 변경해 보기로 했다. 실제로 counter = 1000, counter = 2000과 같이 1,000 단위로 값을 올려서 10,000까지 입력해 보았다.

```
counter = 1000
counter = 2000
// ... repeat in increments of 1000
counter = 10000
```

처음에는 이렇게 클릭 횟수만 10,000으로 바꾸면 결과가 바로 나올 것이라고 생각했다. 그러나 counter를 10,000으로 설정한 뒤 호박을 클릭했을 때, 정상적인 flag가 아닌 이상한 문자들이 화면에 나왔다. 단순히 숫자만 바꾸는 것으로는 정답을 얻을 수 없었다.

그 이유를 알아보기 위해 실제 클릭 이벤트가 처리되는 부분을 자세히 확인했다. 호박을 클릭할 때마다 counter가 1 증가하고, 클릭 횟수가 100의 배수일 때만 pumpkin 배열의 각 원소를 pie 값과 XOR하고 있었다.

```
$('#jack-target').click(function() {
    counter += 1;
    if (counter <= 10000 && counter % 100 == 0) {
        for (var i = 0; i < pumpkin.length; i++) {
            pumpkin[i] ^= pie;
            pie = ((pie ^ 0xff) + (i * 10)) & 0xff;
        }
    }
    make();
});
```

즉, 실제로 중요한 것은 counter 숫자 자체가 아니라, 클릭 이벤트에서 정해진 횟수마다 수행하는 XOR 연산이었다. Console에서 counter = 10000을 입력해도 그동안 수행됐어야 할 XOR 연산이 자동으로 실행되지는 않는다. 오히려 다음 클릭에서는 counter가 10001이 되어 XOR 조건을 지나치고, 아직 변환되지 않은 pumpkin 값이 문자로 표시되므로 이상한 결과가 나온 것이다.

클릭 100번마다 배열 전체에 XOR 연산을 적용하므로 10,000번 클릭하는 동안 이러한 배열 변환은 총 100번 일어난다. 또 pie는 매번 1로 초기화되지 않고 직전 값에서 계속 갱신된다. 여기서 0xff는 255를 뜻하고, & 0xff는 연산 결과의 하위 8비트만 남기는 역할을 한다.

이제 굳이 마우스를 10,000번 누를 필요 없이 이 연산을 그대로 재현하면 되겠다고 생각했고, Python 파일을 작성하기로 했다.

3. Python으로 연산 재현

JavaScript에서 확인한 XOR 연산을 거의 그대로 Python으로 옮겼다. 바깥쪽 반복문을 100번 실행하면 클릭 100번마다 수행되는 변환을 모두 재현할 수 있다.

```
pumpkin = [124, 112, 59, 73, 167, 100, 105, 75, 59, 23, 16,
           181, 165, 104, 43, 49, 118, 71, 112, 169, 43, 53]
pie = 1

for i in range(100):
    for j in range(len(pumpkin)):
        pumpkin[j] ^= pie
        pie = ((pie ^ 0xff) + (j * 10)) & 0xff

hex_string = bytes(pumpkin).hex()
print(hex_string)
```

실행 결과 다음과 같은 16진수 문자열을 얻었다.

44487b495f6c696b335f70756d706b316e5f7069337d

처음에는 16진수 문자열이 출력돼 이것이 답인지 바로 알아보지 못했다. 하지만 해당 값을 바이트로 변환한 뒤 ASCII 문자로 해석하니 flag 형식의 문자열이 나왔다. Python에서는 bytes.fromhex(hex_string).decode()를 이용하면 된다.

DH{I_lik3_pumpk1n_pi3}

실제로 원본 JavaScript의 make() 함수도 pumpkin 배열을 String.fromCharCode()로 문자로 바꿔 출력하고 있었다. 화면 출력 조건은 counter > 10000이므로, 정상적인 클릭 흐름에서는 10,000번째 클릭에서 마지막 XOR 연산을 마치고 10,001번째 클릭부터 flag를 보여준다.

4. 알게 된 점

이번 문제는 단순히 10,000번 클릭하면 끝나는 기초적인 문제처럼 보였지만, 그 안에서 정답이 만들어지는 과정을 직접 확인할 수 있었다. 처음에는 Console에서 counter만 10,000으로 바꾸면 될 것이라고 생각했으나, 실제로는 클릭 이벤트가 실행되면서 pumpkin 배열의 값도 함께 변해야 한다는 것을 알게 되었다.

특히 화면에 표시되는 숫자를 바꾸는 것과 프로그램 내부의 연산을 실행하는 것은 다르다는 점이 기억에 남는다. 문제 자체는 비교적 쉬웠지만, 개발자 도구를 이용해 원인을 찾고 핵심 XOR 로직을 Python으로 옮겨 해결해 본 좋은 연습이었다.